CLAUDE CODE

Présentation de groupe

Plan

01 C'est quoi Claude code ?

02 MCP

03 Skills

04 Prompt engineering

05 Techniques de tool use

06 Environnements de terminal

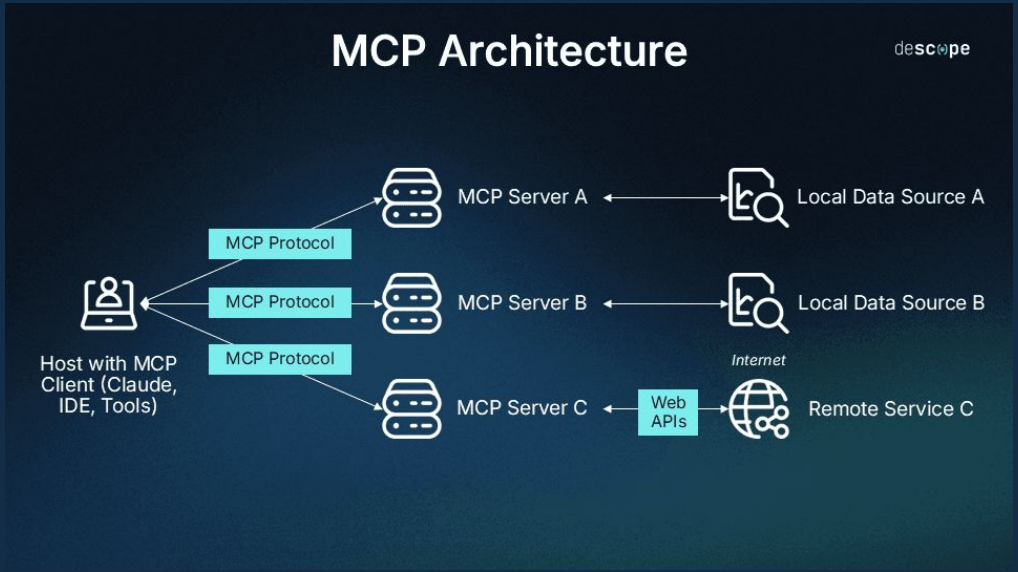
C’est quoi, Claude code ?

MCP

Qu’est-ce qu’un MCP ?

Définition

Standard open-source pour connecter applications IA à des systèmes externes. Propose un langage commun pour que les modèles puissent accéder aux données et exécuter des actions.



MCP

Fonctions principales et exemple concret

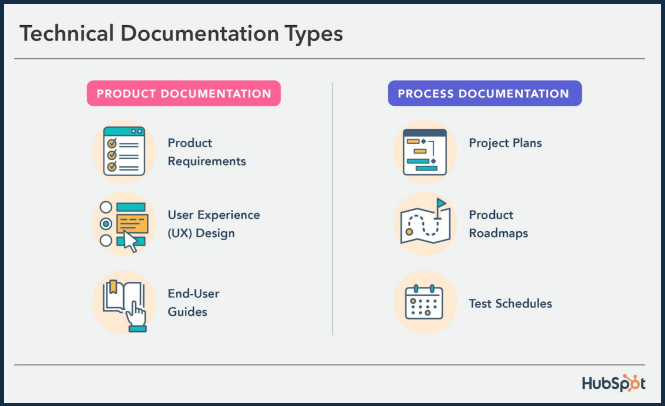
Définition

- Lire ou modifier des fichiers dans un projet,
- Interroger une base de données,
- Appeler une API externe ou
- Encore automatiser certaines tâches de développement.

Exemple au Crédit Agricole Consumer Finance & Mobility

Exposer la documentation externe pour :

- Audition de sécurité et conformité,
- Développer,
- Initialiser des projets,
- Déployer les projets et
- Expertise technique, …



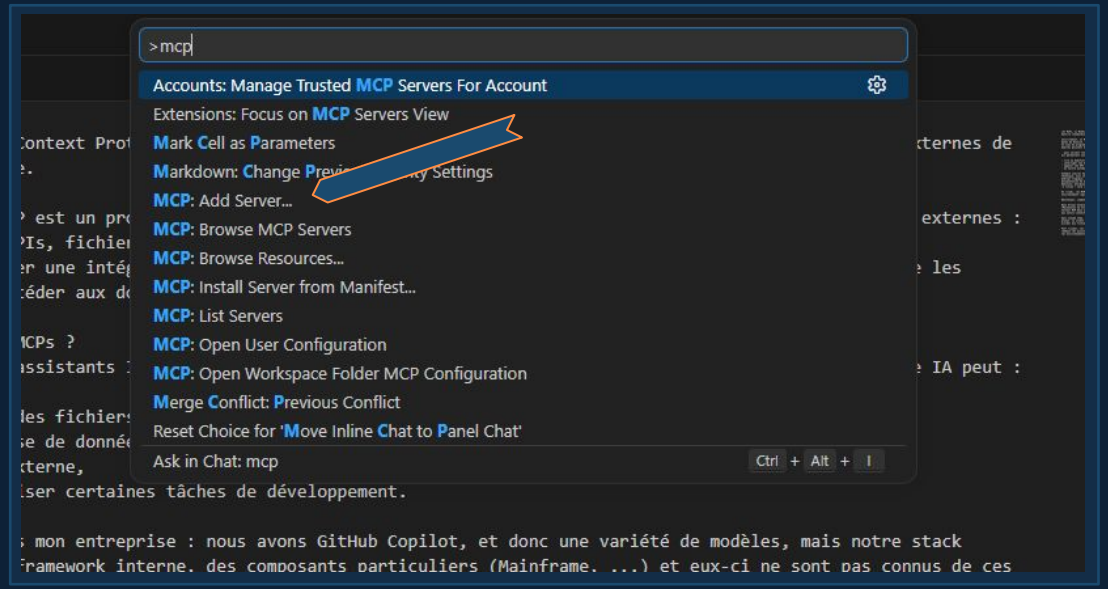
GitHub Copilot

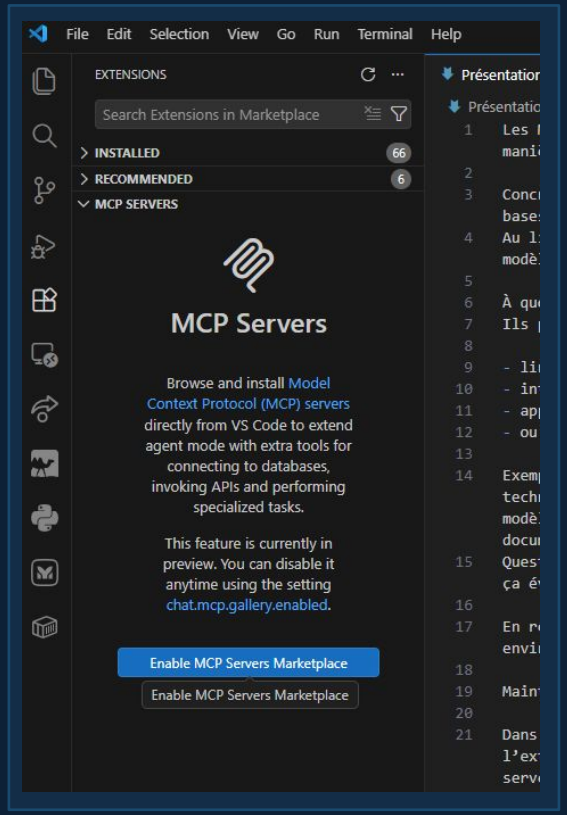
MCP

Tutoriel d'installation (pour VSCode) - 1

Activation du Marketplace

Accéder à la configuration



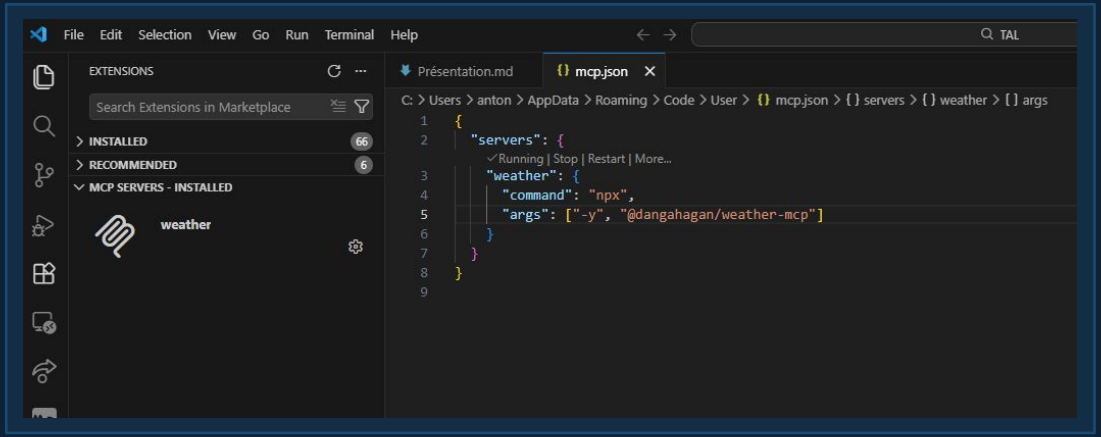


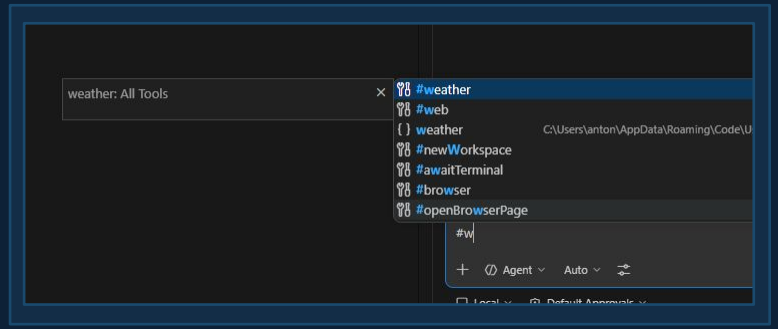
MCP

Tutoriel d'installation (pour VSCode) - 2

Ajout de la configuration (JSON)

Accéder à la configuration



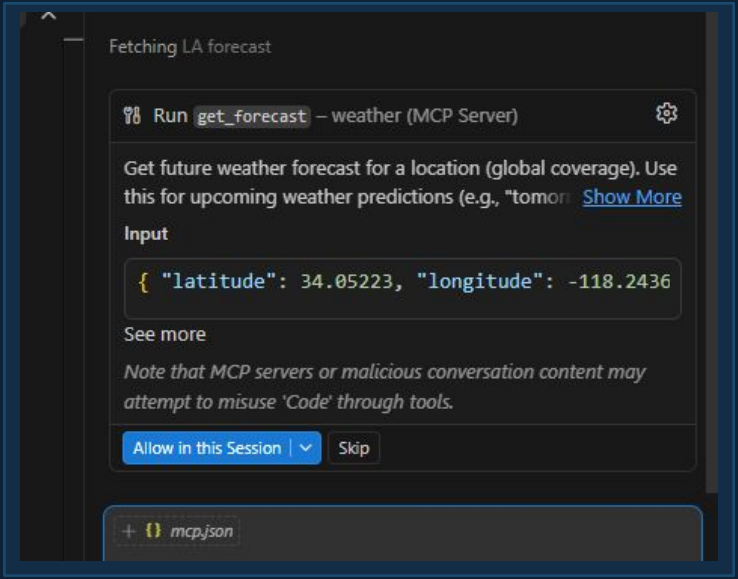


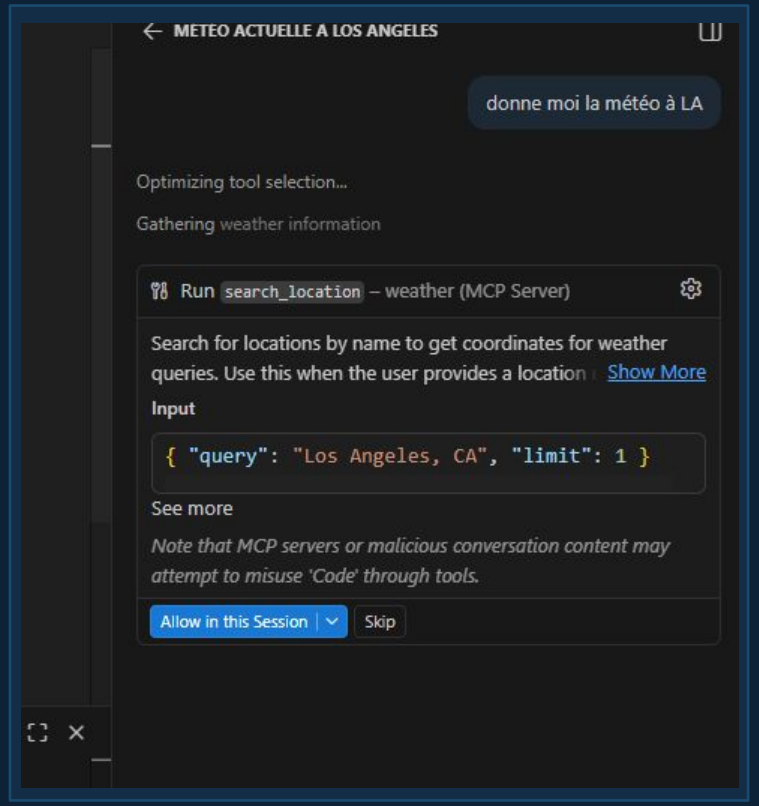
MCP

Tutoriel d'installation (pour VSCode) - 3

Essayer le MCP

Accéder à la configuration

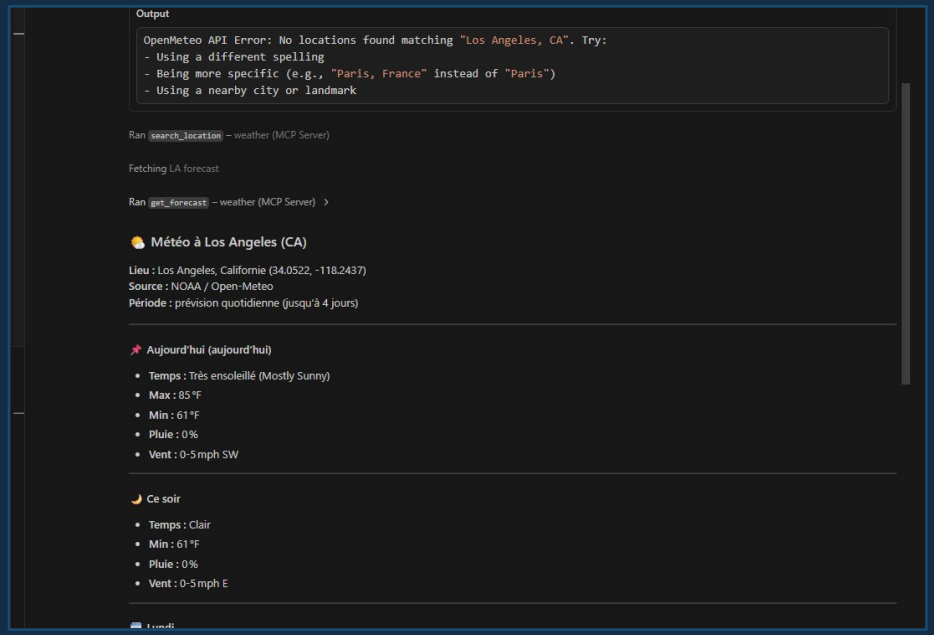




MCP

Tutoriel d'installation (pour VSCode) - 4

On obtient les bon résultats



Trouver des MCP

https://registry.modelcontextprotocol.io/
https://github.com/mcp

et bien d'autres !
https://modelcontextprotocol.info/docs/quickstart/quickstart/ -> tutoriel sur **Claude Caude**
https://www.workato.com/the-connector/mcp-server-tutorial/ -> Installation sur **VSCode**

Qu'est-ce qu'un skill ?

Un skill, c'est quoi ?

Un dossier contenant des instructions, des scripts et des ressources (ex: un fichier SKILL.md).

Objectif ?

Enseigner à Claude comment accomplir une tâche spécifique de manière fiable (génération d'UI, code review, normes de commit, fabrication de documents Word…).

Comment créer et ajouter un skill ?

Exemple : Revue de sécurité du code, avec un format spécifique à un projet

```
NLP
├── .claude
│   ├── settings.local.json
│   └── skills
│       └── sec-review
│           └── SKILL.md
```

~/.claude/ pour un skill global

```
---
name: sec-review
description: Effectue une revue de code orientée sécurité pour identifier les vulnérabilités (injections SQL, secrets en dur, etc.) et propose des cor
rections sécurisées.
---
```

```
# Skill : Revue de Sécurité (AppSec)

Tu agis en tant qu'expert en sécurité applicative. Lorsqu'on fait appel à cette skill (via `/sec-review` ou automatiquement), tu dois analyser le code
 fourni en appliquant strictement le workflow suivant :

## 1. Analyse
Recherche activement les failles de sécurité communes, notamment :
* Injections (SQL, Commandes, etc.)
* Identifiants ou secrets en dur
* Mauvaise gestion des erreurs exposant des données sensibles

## 2. Format de Rapport
Tu dois toujours structurer ta réponse de cette manière, sans texte d'introduction inutile :

* 🚨 **Vulnérabilité :** [Nom de la faille]
* 📍 **Localisation :** [Ligne / Fonction affectée]
* 💥 **Impact :** [Explication très courte du risque]
* 🛡️**Solution :** [Explication de la correction]

## 3. Correction
Après avoir fourni le rapport, génère le code corrigé en utilisant les meilleures pratiques (par exemple, des requêtes paramétrées pour contrer les in
jections SQL). Demande ensuite à l'utilisateur s'il souhaite que tu appliques cette modification directement dans le fichier.
```

Comment utiliser ce skill ?

```
> /sec-review main.py

● Read 1 file (ctrl+o to expand)

● Rapport de Sécurité — main.py

  ---
  - 🚨 Vulnérabilité : Injection SQL
  - 📍 Localisation : Ligne 9 — fonction get_user_data()
  - 💥 Impact : Un attaquant peut manipuler la requête SQL en
  permettant de contourner l'authentification ou d'exfiltrer
  - 🛡️ Solution : Utiliser des requêtes paramétrées (placeho
```

```
> Hey asy fait moi une revu de sécu stp

● Skill(sec-review)
  └ Successfully loaded skill

● Explore(Explore NLP codebase for security review)
  └ Done (7 tool uses · 15.7k tokens · 14s)
  (ctrl+o to expand)

● ---
  Rapport de Revue de Sécurité — main.py

  ---
  🚨 Vulnérabilité : Injection SQL
  📍 Localisation : main.py:9 — fonction get_user_data()
  💥 Impact : Un attaquant peut manipuler la requête SQL pour
   ligne 24 le démontre directement avec admin' OR '1'='1.
  🛡️ Solution : Utiliser des requêtes paramétrées (?) au lieu
```

Quelques skills déjà existants

github.com/heilcheng/awesome-agent-skills

```
> /plugin

Plugins  Discover  Installed  Marketplaces  Errors

Discover plugins

 ⌕ document-sk

  ○ document-skills · anthropic-agent-skills
    Collection of document processing suite including Excel, ...

type to search · Space to toggle · Enter to details · Esc to back
```

Official Claude Skills (Document Processing)

Claude provides built-in skills for common document types:

Skill	Description	Source
docx	Create, edit, analyze Word documents with tracked changes	anthropics/skills
xlsx	Spreadsheet manipulation: formulas, charts, data transformations	anthropics/skills
pptx	Read, generate, and adjust slides, layouts, templates	anthropics/skills
pdf	Extract text, tables, metadata from PDFs	anthropics/skills

Composition de skills

"Do a code review ... make a pptx slide called example.pptx showing how security is amazing"

```
● Skill(sec-review)
  └  Successfully loaded skill

● Explore(Explore Python codebase)
  └  Done (2 tool uses · 14.3k tokens · 6s)
  (ctrl+o to expand)

● Here is the security report for main.py:
```

```
  Now generating the example.pptx presentation based on this report.

● Skill(pptx)
  └  Successfully loaded skill

● Read 1 file (ctrl+o to expand)

● Now I'll create the JavaScript file to generate the presentation.
```

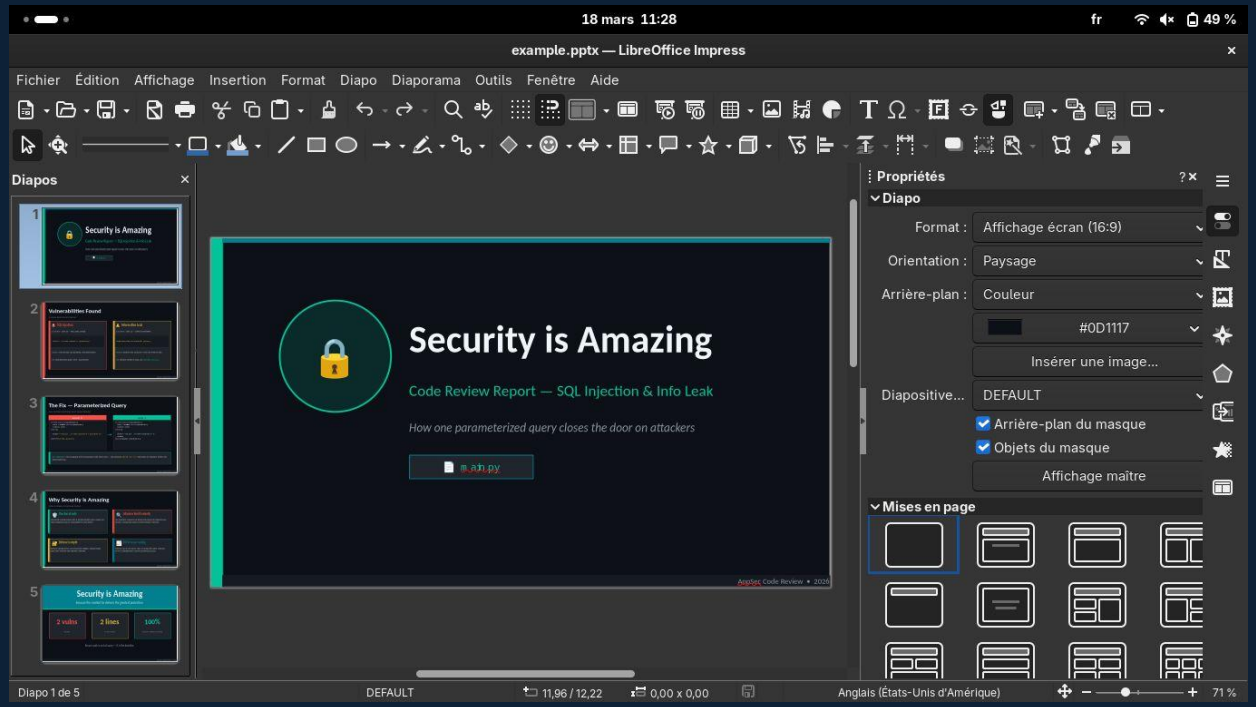
Skills composés

- 1. sec-review
- 2. pptx

Claude enchaîne automatiquement

Composition de skills - résultat

coût : ≈ 0,80 €



Prompt Engineering

Une meilleure entrée pour une meilleure sortie

Instructions

Contexte

Contraintes

Exemples

Prompt Engineering en Analogie

Écrire le code avec l'IA ⟷ Déléguer à quelqu'un

On veut donner un maximum d'informations utiles !

CLAUDE.md

Une documentation pour se familiariser au projet

Contexte et Objectifs

Standards et Workflows

Outils et Frameworks

Exemples

… Et plus !

Comment le créer et l’utiliser ?

```
> /init

/init                    Initialize a new CLAUDE.md file with codebase
                         documentation
```

→ *Explore tous les fichiers du projet pour écrire un première version*

/!\ Bien vérifier le contenu généré

Un exemple de contenu

```
# Project Context

When working with this codebase, prioritize readability over cleverness.
Ask clarifying questions before making architectural changes.

## About This Project

FastAPI REST API for user authentication and profiles.
Uses SQLAlchemy for database operations and Pydantic for validation.

## Key Directories

- `app/models/` - database models
- `app/api/` - route handlers
- `app/core/` - configuration and utilities
```

```
## Standards

- Type hints required on all functions
- pytest for testing (fixtures in `tests/conftest.py`)
- PEP 8 with 100 character lines

## Common Commands
```bash
uvicorn app.main:app --reload  # dev server
pytest tests/ -v               # run tests
```

Notes

All routes use `/api/v1` prefix. JWT tokens expire after 24 hours.
```

Hiérarchie

Tier	macOS/Linux	Windows	Scope	Git Tracked?
Global	`~/.claude/CLAUDE.md`	`%USERPROFILE%\.claude\CLAUDE.md`	All projects, all sessions	❌ No (personal config)
Project	`/project/CLAUDE.md`	`\project\CLAUDE.md`	One repository, all team members	✅ Yes (shared conventions)
Local	`/project/.claude/CLAUDE.md`	`\project\.claude\CLAUDE.md`	One project, one developer	❌ No (.gitignore'd)

Local > Project > Global
→ Le fichier le plus spécifique prend priorité pour une même instruction

1. **Global first:** `~/.claude/CLAUDE.md` → base instructions
2. **Project second:** `/project/CLAUDE.md` → adds/overrides
3. **Local last:** `.claude/CLAUDE.md` → final overrides

L’alliance des deux

CLAUDE.md

- Contexte du projet
- Règles générales

Prompt Engineering

- Contexte de la tâche
- Règles spécifiques

Requêtes plus efficaces

Sorties plus cohérentes

Itération plus rapide

Techniques de Tool Use

C’est quoi le Tool Use ?

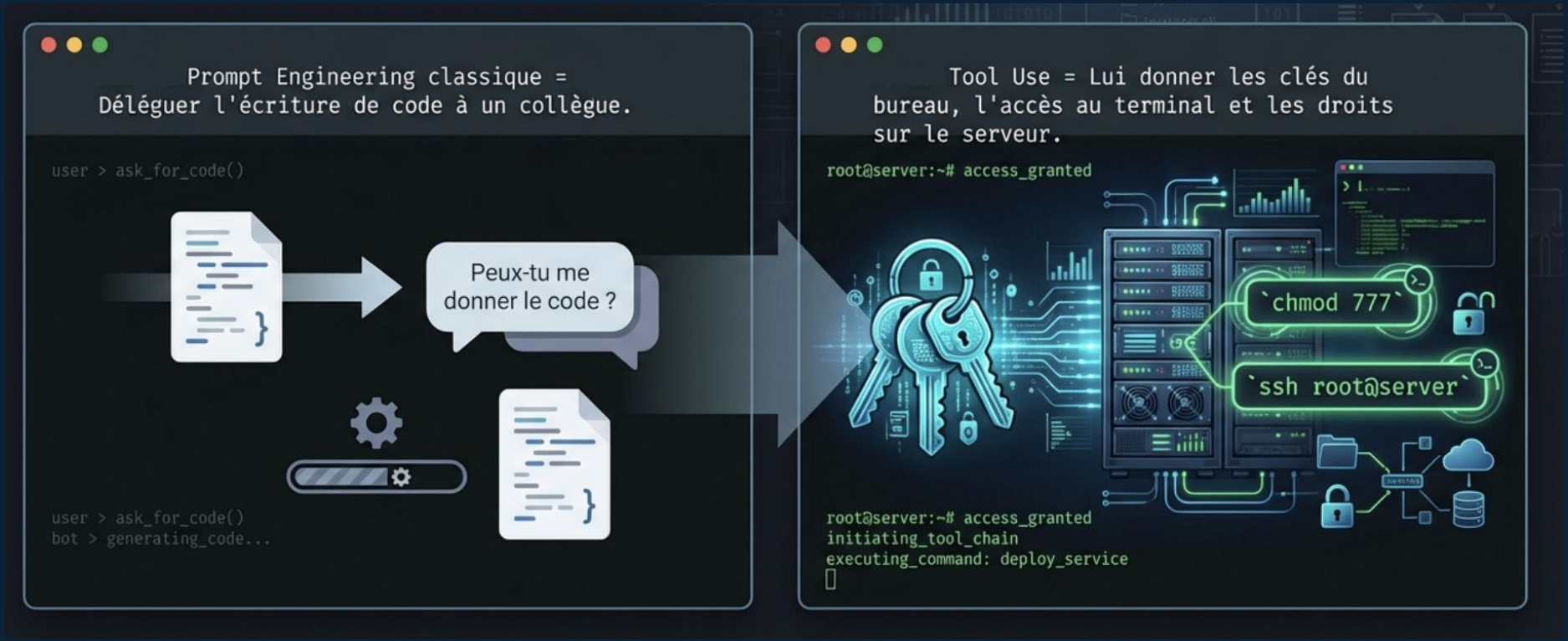
Par défaut : un "cerveau" enfermé dans une boîte de texte (aveugle aux fichiers).

Avec le Tool Use : la capacité d'interagir en direct avec votre environnement.

Donner des “mains” et des “yeux” à Claude

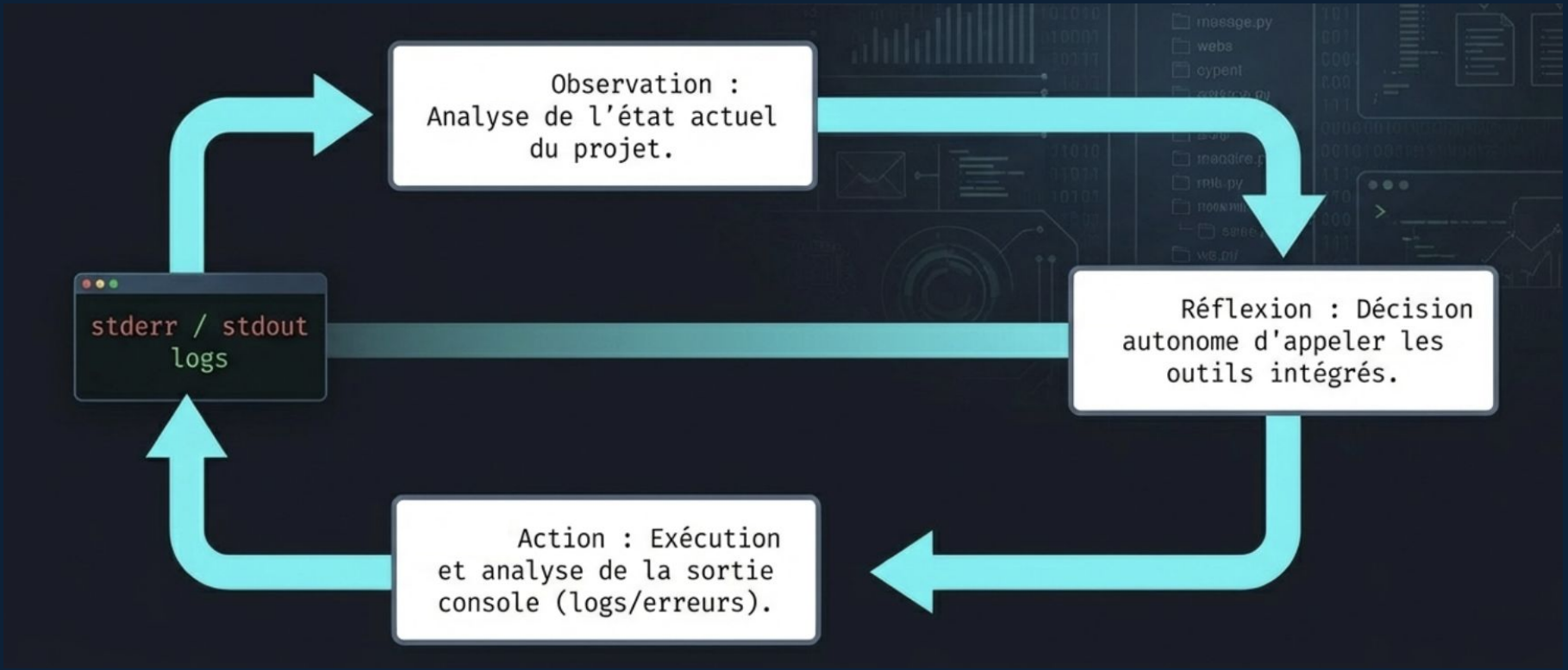
L’analogie de la délégation

Du conseil externe à l’intégration dans l’équipe

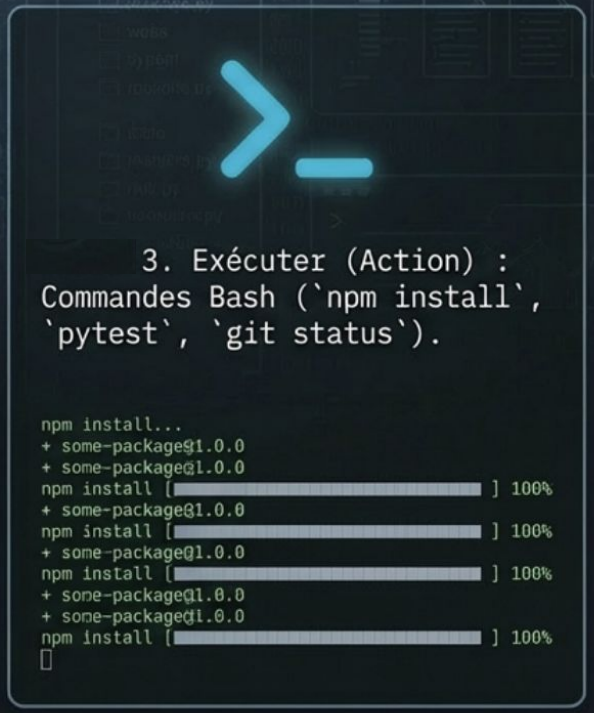


Comment ça fonctionne concrètement ?

La boucle itérative

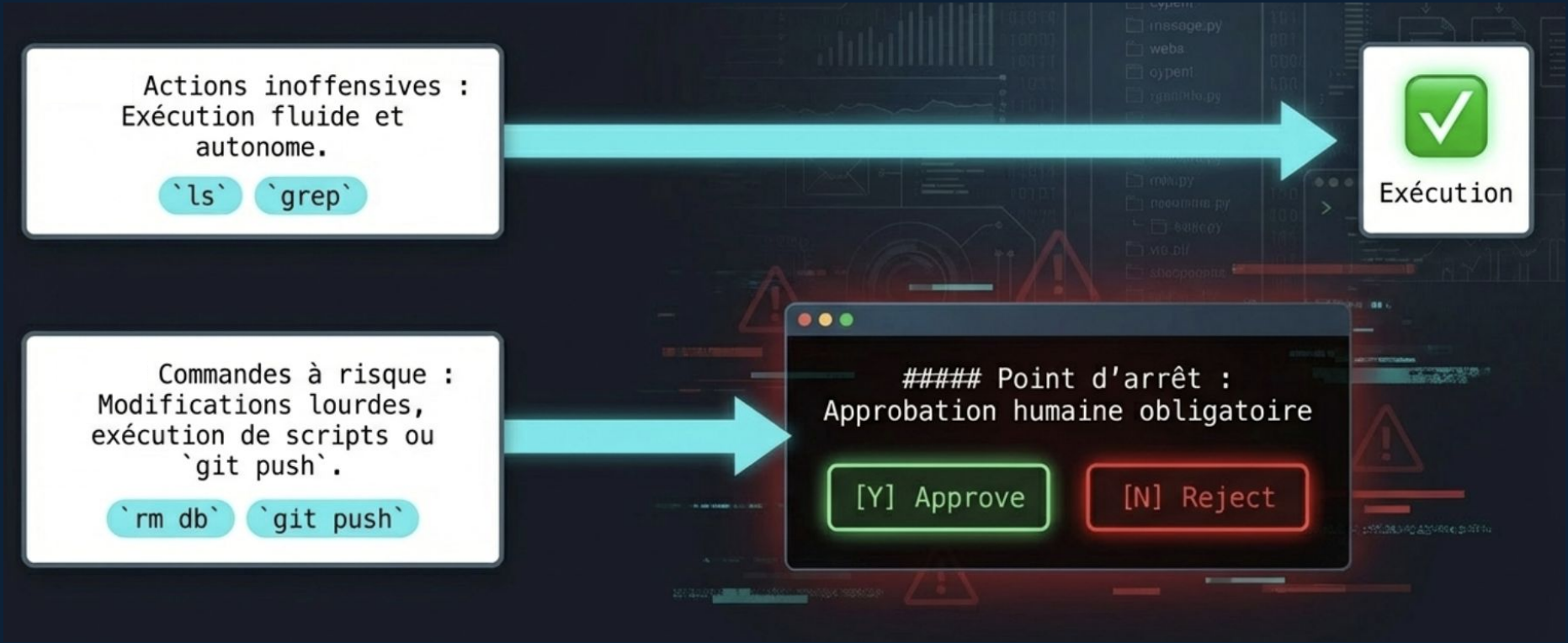


Exploration > Edition > Exécution



Sécurité & Contrôle (Human-in-the-loop)

Est-ce dangereux de laisser une IA taper des commandes dans mon terminal ?



Techniques pratiques pour optimiser le Tool Use

La boucle itérative

Actions de bout en bout :

~~Écris-moi une fonction d'authentification.~~

Implémente l'authentification dans auth.ts, puis lance les tests unitaires associés pour vérifier que ça marche.

L'IA mène l'enquête :

~~Comment corriger cette erreur ?~~

J'ai l'erreur [log] quand je lance l'app. Trouve d'où ça vient et propose une correction.

Automatisation :

~~Comment faire un rechercher/remplacer partout ?~~

Renomme la variable X en Y dans tout le projet et mets à jour la documentation en conséquence.

En résumé…

Le Générateur de Code	Presse-papier manuel, aveugle au contexte, correction dirigée par l'humain.
L'Agent Autonome	Navigation autonome, conscient du projet, auto-correctif via les logs.

ENV TERMINAL

Claude Code ≠ chatbot

Claude Code s'exécute dans un vrai shell - l'environnement conditionne tout ce qu'il peut faire.

Machine locale

Accès direct aux fichiers, outils installés. Idéal pour le dev.

Container / CI

Environnement isolé, reproductible. Parfait pour l'automatisation.

VM / Proxmox

Snapshots, isolation réseau. Claude tourne dans une VM dédiée.

SSH distant

Claude se connecte à un serveur distant. tmux garde la session active.

Ce que ca change : permissions · accès réseau · persistance des fichiers · outils disponibles

Pourquoi tmux ?

Claude peut lancer des processus longs - il a besoin de plusieurs mains.

```
[0] claude  [1] server  [2] logs      tmux  18/03
```

PANE 1L - Claude

```
$ claude

Claude: Je lance
le serveur de dev...

Editing App.jsx...
Running tests...
```

PANEL 2 - Server

```
$ npm run dev

  Local: localhost:3000
  watching...

  compiled
  compiled
  compiled
```

Sessions persistantes

La session survit à une coupure SSH ou à une fermeture de terminal.

Processus longs

npm run dev tourne en parallèle pendant que Claude code dans un autre panel.

Multi-panels

Claude observe les logs en temps réel sans perdre son contexte de travail.

Bonnes pratiques & sécurité

Donner à Claude un environnement isolé et bien contrôlé.

Pas root

Faire tourner Claude sous un user limite, jamais en root.

Isoler

Container Docker ou VM dédiée - les dégâts restent confinés.

Logs visibles

Garder les logs accessibles pour comprendre ce que Claude a fait.

Fichiers organisés

Structure claire : Claude s'y retrouve mieux, moins d'erreurs.

/allowed-tools

Restreindre ce que Claude peut faire :

```
# Autoriser seulement :
Bash(git:*)
Bash(npm run *)
Read(*)
Edit(src/**)

# Interdire :
Bash(rm -rf *)
Bash(sudo *)
WebFetch(*)
```

Principe : least privilege pour un agent autonome